Mark I Core: component-wise build path (technical plan)

Context

Vatsal wants FRED rebuilt from the ground up in `Mark_I/Initial/Core/`, fully local, built component by component like the memory system, following the assembled architecture (L0 senses, L1 interaction clock, L2 talker/voice, L3 thinker, L4 proactivity, L5 consolidation) and using `Old/Core` as reference. The shortcomings found in the log review (`Mark_I/Documentation/2026/10/06/fred_log_review.md`) are fixed by design inside the replacing component, not by patching the old code. This plan covers technicalities only; scope, network tools, first tool set, brain choice and data location are decided later and do not block P0. Nothing is built until this plan is approved.

Method (same as memory): per step, only the affected components, each with sockets in its docstring, a runnable self-check (`python <file>.py`), tests with no GPU where possible, docs in `Mark_I/Documentation/2026/10/NN/`, commit and push after each step, no Claude co-author trailer. Current FRED stays untouched and down. Fine-tunes come last.

Verified starting facts

- System Python 3.10.11 has numpy, fastapi/uvicorn, sounddevice, PyAudio, pywin32, pycaw, mss, opencv, psutil, requests, pytest, torch 2.14. It lacks faster-whisper, onnxruntime, kokoro-onnx, openwakeword, speechbrain, insightface, webrtcvad. `Old/Core/venv` has them (torch 2.11+cu128, onnxruntime 1.27, faster-whisper 1.2.1, kokoro-onnx 0.5.0, openwakeword 0.6.0, speechbrain 1.1.0, webrtcvad-wheels, insightface, llama_cpp_python 0.3.31).
- Playground (`Mark_I/Initial/playground/server.py`) has two slots (chat 8191, embed 8192), both fully on GPU, vision via `mmproj`. The assembled design needs a third resident process (talker/judge, own llama-server) and a describer, so the slot table must grow (see P0).
- Old tool loop: `MAX_TOOL_ROUNDS = 4`, `_generate_with_tools` (orchestrator.py:2812), text tool-call parser (`_parse_text_tool_calls`, :3071), reply guards (`_claims_completed_action`, `_unsupported_claim`, `_looks_like_leaked_tool_syntax`), confirmation gate (`_run_or_confirm`, `_request_confirmation`), classifier (`_classify_turn`), router (`orchestrator/intent.py`, `tool_router.py`), dispatcher (`orchestrator/dispatcher.py`). These are the logic to port in P1.
- GPU now: 1.9 GB used of 16.3 GB (FRED is down).

Layout (`Mark_I/Initial/Core/`, one file per job, no class over ~400 lines)

```
config.py          typed settings from code defaults + FRED_DATA dir (outside repo) + env
overrides
log.py             three streams: session.jsonl (real), tools.jsonl, crash (faulthandler); tests
write to a temp dir
events.py          Event(ts, source, kind, data) + EventBus + EventLog (JSONL ring ~10 min,
persisted daily)   [L4 input, memory Archive feed]
gpu.py             GPU scheduler for in-process GPU users (whisper, presence, vision-local): one
queue, priorities, timeouts
models/
  client.py        playground client: chat(messages, tools, schema, effort, stream), embed,
vision, load/unload, health
  fake.py          scripted fake LLM + fake embedder (tests with no GPU)
  toolparse.py     per-family tool-call parsers (native tool_calls, Qwen XML/Hermes text,
Pythonic [f(a=..)], MiniCPM XML)
tools/
  registry.py      ToolSpec(name, fn, schema, group, destructive, timeout_s, reads_untrusted,
sends); Result(ok, data, error_kind, speakable)
  coerce.py        validate + coerce arguments from the JSON schema before the call (int->str,
env vars, relative paths)
  <area>.py        one file per area: time, calc, audio_dev, system, files, windows, agenda,
reminders, power, vision, memory (wraps memory/memtools.py), ...
engine/
  turn.py          Turn(text, source, interrupted_at, ts); EngineEvent stream: ack, tool_start,
tool_result, sentence, done, fault
  dispatch.py      regex fast path, no model (port old dispatcher regexes + confirmations)
  classify.py      cue categories + embedding ranker (port intent.py / tool_router.py), chain-
aware offer
  loop.py          the tool loop and its rules (below)
  gates.py         confirmation gate, read/send separation, agent allowlists, reply guards
  prompt.py        system prompt + message building (+ memory recall socket)
audio/             mic.py (single stream owner), wakeword.py (frozen model), vad.py, stt.py
(windowed faster-whisper), tts.py (Kokoro, speak(cancel=)), phrases.py, devices.py
clock/             turn_controller.py (160 ms tick:
LISTEN/END_OF_TURN/BARGE_IN/BACKCHANNEL/IGNORE), smart_turn.py, voice_id.py
senses/            screen.py (1 fps tile-max change detector + crop), ocr.py (Windows.Media.Ocr,
off until approved), describe.py (vision slot), presence.py, camera.py
proactive/         judge.py (L4), gates.py, scheduler.py (reminders, timers, file watches),
checks.py
memory_bridge/     feed.py (events -> Archive.write), recall.py (judged_search +
PersonalModel.relevant), jobs.py (nightly/weekly/... runners + GPU window), ask.py (askqueue
socket)
ui/                pill.py (Win32 layered window), hud_server.py, popup.py (ask queue text box)
launcher.py        starts playground, waits healthy, starts main; supervises; unloads live stack
for L5 windows
tests/             per-module self-checks stay in the module; cross-module tests + acceptance
suite here
```

Contracts fixed in P0 (everything else plugs into these)

1. **Tool contract:** `ToolSpec` + `Result`. The engine never sees a raw exception or traceback; `error_kind` in {bad_args, not_found, denied, timeout, unavailable, failed}; `speakable` is what FRED may say. Every tool runs in a worker with a timeout; risky ones (file search, subprocess) run in a child process with a time and depth bound.
2. **Model contract:** `Chat.complete(messages, tools=None, schema=None, effort=None, stream=False)` returns text plus parsed tool calls through `toolparse`; same call shapes the memory side already needs (`llm(prompt)`, `llm(messages)`).

3. **Event contract:** every producer emits `Event`; subscribers are the EventLog (L4 input), the memory feed (`Archive.write(stream, data, session, ts)`), the session log and the UI. No component calls another's internals to report something.
4. **GPU contract:** in-process GPU work goes through `gpu.run(owner, priority, fn, timeout)`; llama-server processes are serialised by the playground. Presence/screen/STT can no longer collide with each other or the brain.
5. **Config contract:** defaults in code, `FRED_DATA` (outside the repo) for state, secrets and personal data; tests set `FRED_DATA` to a temp dir so test output never reaches real logs.

The tool loop (L3 engine, fixes the log findings)

- Coerce and validate arguments from the schema before the call (fixes `int.strip`).
- Per turn: refuse an identical call (tool + canonical args) that already failed, returning the earlier error; after 2 failures on one tool stop that tool and give the model a `fault` result so the reply states what failed; hard cap on rounds (start at 4, configurable) and on calls per round.
- Offer by chain: first offer from cue + embedding; if the request has two parts or the model asks for another group, widen to the union of groups (never the full menu by default); log offered vs called.
- Parallel calls: independent calls in one model message run together; dependent ones in order.
- Reply guards ported: false completion (no success evidence from this turn's tool results), leaked tool syntax, unsupported claim; plus a new rule: a failed tool result must be reflected in the reply.
- Streaming: sentences go to TTS as they finish; thinking cap through the server (`chat_template_kwargs` / reasoning budget; verify the llama-server flag in b11433).

Phases in layer order (each shippable, tested, committed before the next; exit gate listed)

Order rule (Vatsal, 2026-10-06): build bottom-up by layer, L0 to L5. Every layer is built and tested against recorded input and a scripted stand-in for whatever sits above it. One deliberate exception: L3 is built right after L1 (before L2) so the voice loop closes end to end as early as possible; L2 then upgrades the output side.

P0 Foundations. `config`, `log`, `events`, `gpu`, `models/client`, `models/fake`, `toolparse`, `tools/registry`, `tools/coerce`. Playground change: generalise slots (chat, embed, talk, vision/describer) with a port table and the "unload live stack, load job model, reload" window for L5. *Gate:* no-GPU self-checks pass; client round-trips chat, schema-forced JSON and embeddings against the real playground; registry rejects/coerces bad args.

P1 L0 Senses. Stand-in above: everything is emitted as `Event`s into the EventLog; replay harnesses feed recorded wavs and the 12 screenshots.

- *1a audio:* `audio/mic` (one stream owner, 16 kHz, 80 ms chunks, fan-out), `wakeword` (loads the frozen model, never swapped), `vad` (webrtcvad, Silero socket), windowed `stt` (faster-whisper through `gpu.run`; STT is unplaced in the layer docs, placed here for the build), hotkey.
- *1b screen:* `screen` (mss 1 fps, tile-max change 0.990, native-res crop), `describe` via the vision slot, `ocr` (Windows.Media.Ocr behind a flag until Vatsal approves CPU use). Grounding rule enforced in code: describer text must appear in the OCR output.

- *1c camera:* `presence` / `camera` (black-frame detection, retry once, then a fault event), all GPU work through `gpu.run`. *Gate:* wake word and VAD events match labelled recordings; STT transcript events on a recorded set; 12-screenshot set passes; presence runs 8 h with no CUDA errors in the log.

P2 L1 Interaction clock. `clock/turn_controller` (160 ms tick, states LISTEN / END_OF_TURN / BARGE_IN / BACKCHANNEL / IGNORE), `smart_turn`, `voice_id`, echo handling. Rules: end of turn = silence >= 200 ms and Smart Turn >= 0.5, or 1.2 s hard cap, or chord release; barge-in = speech >= 300 ms and voice ID and (headphones or level above echo floor). Stand-in thinker: echoes the transcript. Offline harness first, on recorded wavs, before the live mic. *Gate:* offline clock decisions match labelled recordings (mid-sentence pauses, short commands, backchannels); live hold-to-talk and wake loop returns the echo; thresholds are settings, not literals.

P3 L3 Thinker (built before L2 so the loop closes; output is text/CLI until P4).

- *3a engine:* `engine/*`, first tools (port of the sound old tools: time, calculator, volume/mute/devices, reminders/timers/agenda, files read/write/search, windows, lockdown/power, screen/camera tools over the P1 events), a CLI, then wired to the P2 turn stream. Port the old benchmark cases (`Old/benchmarking/cases/*.jsonl`, tools recorded not executed) as `tests/acceptance`. *Gate:* scripted-model tests cover every loop rule (identical failure refused, 2-failure stop, chain offer, parallel, guards); acceptance at or above the old baseline (tools 77.8%, conversation 76.2%) on the chosen brain; zero identical failed repeats in the run logs.
- *3b memory bridge:* `memory_bridge/*` per integration plan steps 1-5 (feed, recall in turn with "digging deeper" via a `say` hook, models, timers, spoken memory commands). *Gate:* turns land in the archive with the right streams; recall changes an answer in a scripted test.
- *3c remaining tools:* one group at a time, each with schema, coercion, bounds and a recorded-call test; `launch_application` and `find_file_smart` rebuilt on an app index and a bounded name search. Network/phone/Gmail/smart-AC groups only after their scope decision.

P4 L2 Talker and voice. `audio/tts` (Kokoro, `speak(cancel=)`), `phrases` (phrase cache, fillers, tool captions fed by `tool_start`), `devices`; talker slot wired but optional (stand-in: phrase cache only; 1B vs 2B decided later). Barge-in from P2 now cancels real speech. *Gate:* sentence streaming; cancel < 200 ms measured; phrase-cache coverage reported on logged tool turns.

P5 UI. `ui/pill` (port the Win32 layered window from old `ui/pill_app.py`, 57 KB, rebuilt around the event bus), `hud_server` (port `hud/`), `popup` for the ask queue (integration step 6). *Gate:* `--mock` mode cycles all states; a real turn shows state and text; a popup reply reaches `askqueue.answer`.

P6 L4 Proactivity. `proactive/judge` in shadow mode (log only, 2-3 days) on the EventLog, gates and cooldowns ported from `proactive_checks`, `scheduler` for reminders/timers/watches, `memory_bridge/ask` (conversation-end trigger, precedents recorder), digest and alerts spoken on request. *Gate:* shadow log reviewed; judge JSON schema-forced; nothing speaks except L4 and replies.

P7 L5 Consolidation. `memory_bridge/jobs`: nightly/weekly/monthly/yearly runners, maintenance, backup, alerts and self-analysis timers through the GPU window (launcher unloads the live stack, loads the job model, reloads; skipped while Vatsal is present). *Gate:* the nightly and weekly runners complete through the window on the playground and the live stack comes back within the cold-start budget.

P8 Soak and cut-over. 8 h run with flat VRAM, all acceptance and live metrics from the Mark II master plan section 7, then extract `Initial/Core` to `Mark_I/Core`. Tuning (thresholds, offer sizes, brain choice, talker 1B vs 2B, thinking cap) happens here with the finished stack.

Dependencies and installs

- Per phase only what it needs, listed in `Core/requirements.txt` with the phase tag; no venv per the "no complexity" rule. Open: the system Python lacks the audio/ML packages and is 3.10 (the old venv has them, Python version to confirm). Decide before P1: install into the system Python (check `torch.cuda.is_available()` and ctranslate2 CUDA there first) or run Core on the old venv's interpreter.
- GPU-only rule applies to every model; CPU only for Kokoro, wake word, VAD, Smart Turn, voice ID, Nemotron (per standing permissions). Windows OCR stays flagged off until approved.

Process model

`launcher.py` starts the playground, waits for `/api/status`, starts the main process (engine, audio, clock, senses, UI), supervises with restart on crash (faulthandler log kept). Models: brain in the chat slot, embedder in embed, talker/judge and describer in their own slots, Whisper in the main process under `gpu.run`. L5 jobs: launcher unloads the live stack through `/api/unload`, loads the job model, runs, reloads the stack; skipped while Vatsal is present.

Verification

- Every module: `python <module>.py` self-check (memory convention), no GPU where avoidable; run all after each commit.
- Engine: scripted fake model drives each loop rule; replay of real failing log cases (lockdown int arg, repeated `read_file`, black camera frame) as regression tests with invented content only.
- Acceptance: ported benchmark cases through the engine; compare with old baseline; re-run flipped cases 3x per side.
- Voice: offline wav harness for the clock; live metrics script for barge-in, first word, answered rate.
- Logs: assert tests never write to the real `FRED_DATA`.
- Docs after every step in `Mark_I/Documentation/2026/10/NN/`, README row, commit and push; no personal content in code, tests or docs.

Open technical checks (do not block P0)

1. Python/interpreter and CUDA of the Core runtime (above).
2. llama-server b11433: reasoning-budget flag, tool-call output for Qwen3.5/gpt-oss/LFM/MiniCPM templates (decides which `toolparse` families are needed first).
3. Third and fourth playground slots and their VRAM with the brain resident (budget: brain 5.0-5.6, talker 1.1-2.1, embedder 1.4, whisper 1.6, describer 1.9).
4. MiniCPM-V 4.6, GLM-OCR, Smart Turn, Silero in this stack (L0/L1 open checks); stand-ins keep P3-P6 unblocked.
5. Windows OCR approval; event-log retention and exclusions before L4 runs on real use.